

Montador e Pré-processador NASM

Macros Multilinhas e Depuração com GDB

GRUPO 7

Alan · Roberto Ribeiro · Thiago Fernandes · Eric · Luan Ribeiro

O Pré-processor do NASM

Atuação em tempo de compilação — antes da montagem do código binário.



Substituição Textual

Expande macros e diretivas sem executar o código.



Diretivas com %

Toda diretiva começa obrigatoriamente com %.



Modularidade

Melhora legibilidade, reuso e organização do Assembly.

Macros Multilinhas: Recursos Básicos

Recursos essenciais do arquivo `macros.asm`.



%macro vs %imacro

%macro diferencia maiúsculas;
%imacro as ignora.



Repetição (%rep)

Desenrola loops no
pré-processador, eliminando
overhead em runtime.



Labels Locais (%%)

Gera nomes únicos (ex:
..`@1.fim`), evitando colisões.

Macros Multilinhas: Recursos Avançados

Expandindo o pré-processador ao limite.



Parâmetro Guloso (2+)

Une múltiplos parâmetros em um único argumento.



Defaults (1-3 defaults)

Valores padrão para parâmetros omitidos na chamada.



%0 e %rotate

Processa número variável de parâmetros (ex: `multipush`).



Condition Codes (%+1 / %-1)

Customiza instruções de salto condicional em tempo de compilação.



Sub-listas e .nolist

Passa parâmetros com `{ }` e suprime a listagem de expansão.

Demonstração: Expansão de Macros

Código original vs. código expandido com `nasm -E`

Chamada Original

```
%macro print_char_rep 2
    %rep %2
        print %1, 1
    %endrep
    print newline, 1
%endmacro

print_char_rep char_rep, 3
```

Expansão Gerada (nasm -E)

```
; Repete 3 vezes a syscall:
mov rax, 1 / mov rdi, 1 ...
mov rax, 1 / mov rdi, 1 ...
mov rax, 1 / mov rdi, 1 ...

; Finaliza com newline:
mov rax, 1 / mov rdi, 1 / mov rsi, newline ...
```



O `nasm -E` é uma ferramenta poderosa para depuração, revelando exatamente como o pré-processador transforma seu código antes da montagem final.

Tarefa 1: Programa debug_demo.asm

Desenhado para análise passo a passo no GDB — registradores, pilha, loops e syscalls.



Registradores

Operações add e sub para observar alterações em tempo real.



Pilha (Stack)

push e pop explícitos para verificar modificações do RSP.



Loops

Saltos condicionais com dec, cmp e jg.



Syscalls

sys_write para saída e sys_exit para encerramento limpo.

Depuração com GDB: Comandos Essenciais

Comandos para abrir a "caixa preta" do processador.

```
(gdb) break _start
(gdb) run
Breakpoint 1, _start () at debug_demo.asm:19
19 mov rax, 10
(gdb) nexti
20 mov rbx, 5
(gdb) info registers rax
rax 0xa 10
(gdb) nexti
... (executa add, sub e pushes até linha 30)
...
30 mov rax, 0
(gdb) x/1xg $rsp
0x7fffffff3b0: 0x0000000000000005
(gdb) x/1xg $rsp+8
0x7fffffff3b8: 0x000000000000000d
```

01

Inicialização

`gdb ./debug_demo` e `break _start`

02

Avanço

`nexti` (ou `ni`): instrução por instrução.

03

Registradores

`info registers rax rbx`: mudanças nos registradores.

04

Memória e Valores

`x/1xg $rsp` examina a pilha; `print $rdx` mostra o acumulador.

debug_demo.asm: Aritmética e Pilha

Seções 1 e 2 — operações com registradores e manipulação da stack.

Operações Aritméticas

```
mov rax, 10 ; rax = 10
mov rbx, 5 ; rbx = 5
add rax, rbx ; rax = 15
sub rax, 2 ; rax = 13
```

Uso da Pilha (LIFO)

```
push rax ; empilha 13
push rbx ; empilha 5
mov rax, 0 ; zera rax
pop rbx ; rbx = 5
pop rax ; rax = 13
```



GDB: `info registers rsp` para rastrear o ponteiro da pilha.

debug_demo.asm: Loop e Syscalls

Seções 3, 4 e 5 — laço de soma, saída e encerramento.

Loop de Soma

```
mov rcx, 5 ; contador
mov rdx, 0 ; acumulador
laco_soma:
    add rdx, rcx ; rdx += rcx
    dec rcx ; rcx--
    cmp rcx, 0
    jg laco_soma ; se rcx > 0, repete
```

Syscalls

```
; sys_write
mov rax, 1
mov rdi, 1
mov rsi, msg
mov rdx, len
syscall

; sys_exit
mov rax, 60
xor rdi, rdi
syscall
```



As syscalls permitem que o programa peça serviços ao kernel, como escrita e encerramento.